

Paradigmas e Linguagens de Programação

Aula 10 **CUDA** **(Compute Unified Device Architecture)**

- **Introdução**
 - **Características**
- **Por que usar CUDA**
- **Arquitetura CUDA - Conceitos básicos**
- **Ferramentas e Desenvolvimento**
- **Etapas de um programa CUDA típico**
- **Limitações e Desafios**
- **Material para consulta/estudo de CUDA**

Introdução

- **CUDA (Compute Unified Device Architecture):**
- É uma **plataforma de computação paralela desenvolvida pela NVIDIA**. Ela permite que programadores utilizem a potência de **processamento das GPUs** para resolver **problemas computacionais complexos** que normalmente seriam executados na CPU.
- **Características:**
 - Usa extensões das linguagens C/C++ para programar a GPU.
 - Possibilita **escrever funções chamadas de kernels** que são executadas por **milhares de threads simultaneamente**.
 - Altamente escalável: pode ser usada para problemas simples ou extremamente grandes.



Por que usar CUDA

CPUs são excelentes para tarefas sequenciais e controle de fluxo, mas têm poucos núcleos. Já **GPUs** têm **centenas ou milhares de núcleos** que executam tarefas **em paralelo** com muita eficiência.

Vantagens:

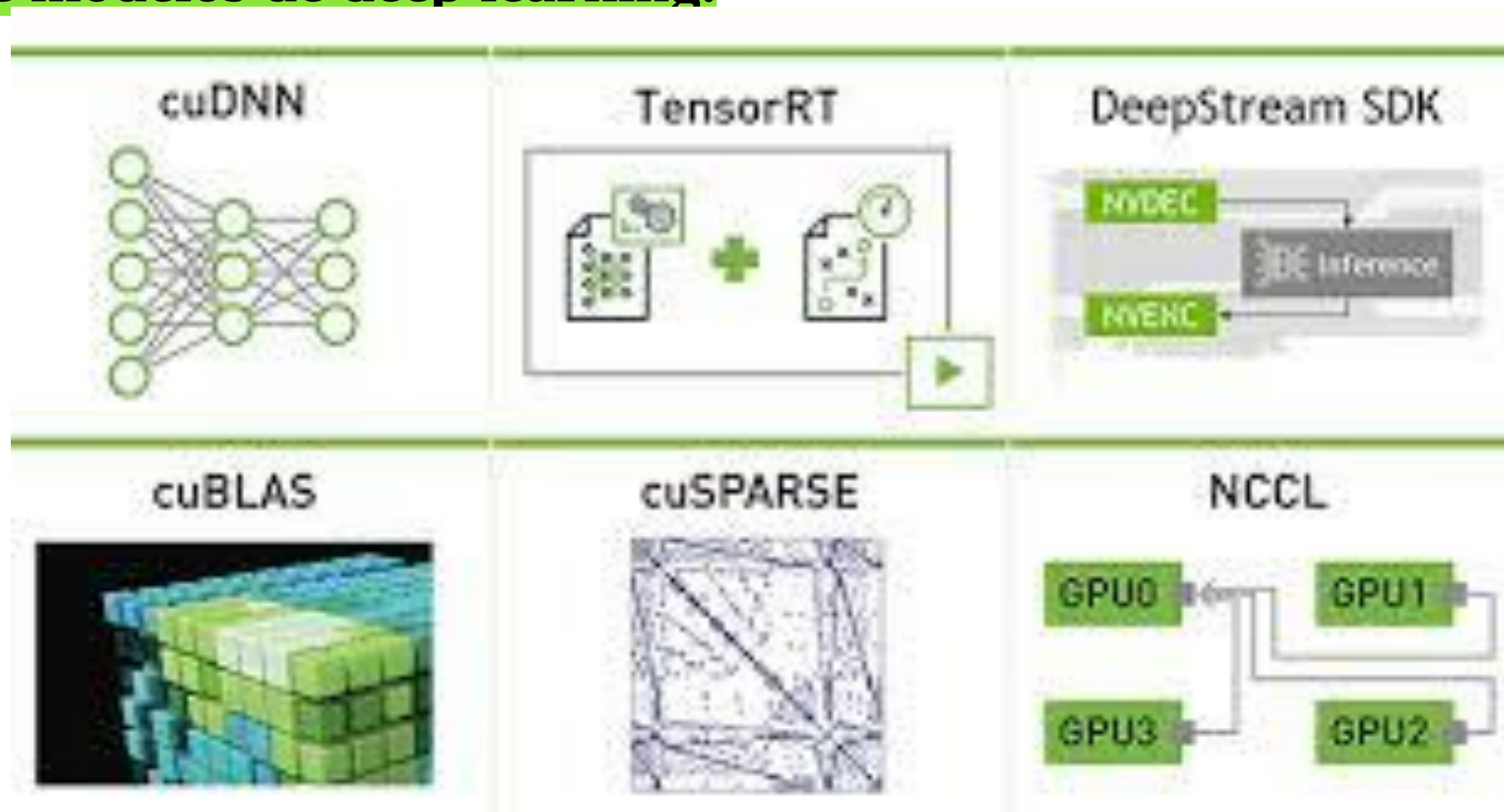
- Grande **aumento de desempenho** em tarefas paralelizáveis (até 10x ou mais).
- Redução de tempo de execução de aplicações de IA, ciência de dados, simulações físicas, etc.
- Excelente suporte pela NVIDIA, com ferramentas de desenvolvimento, profiling e bibliotecas otimizadas.



CUDA

Exemplos de Uso:

- **Treinamento de modelos de deep learning.**



CUDA

Exemplos de uso real:

- **Renderização gráfica em tempo real.**

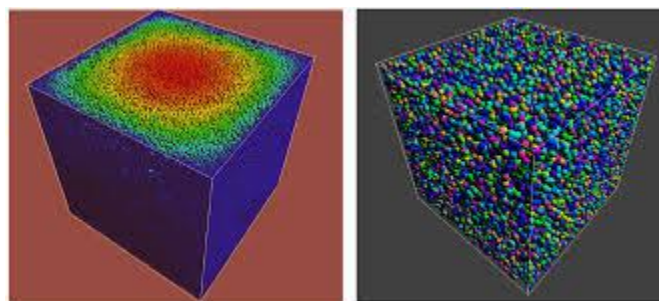
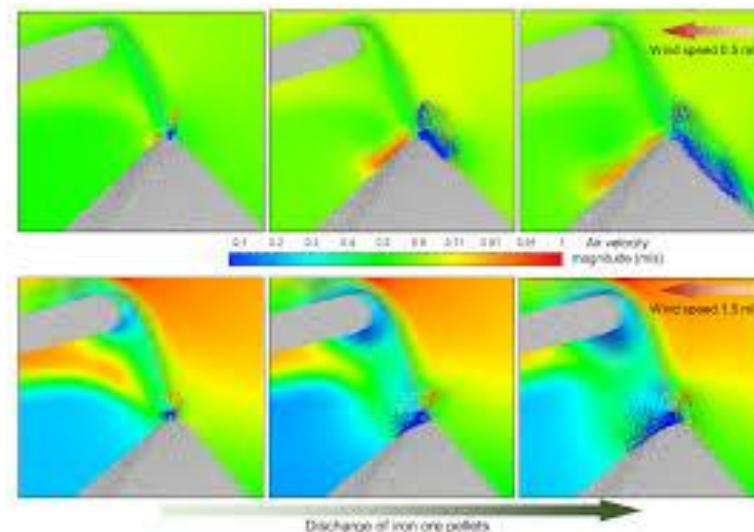
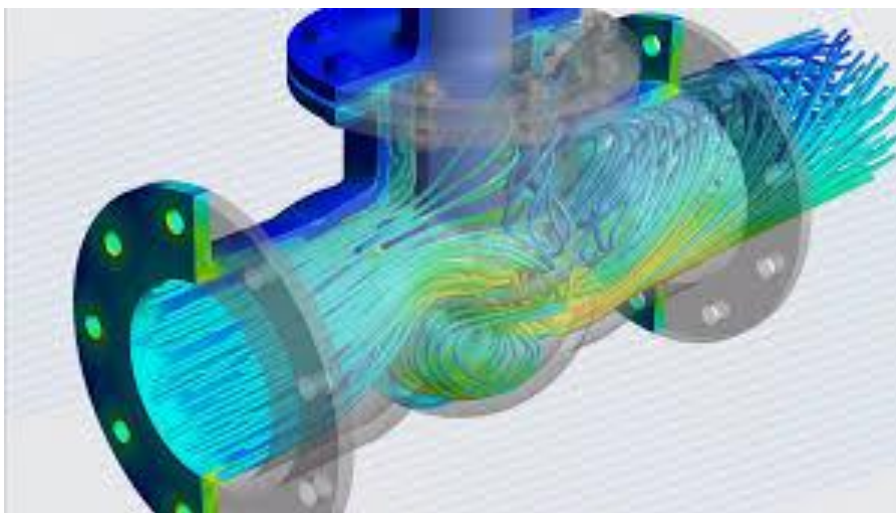


<https://www.nvidia.com/pt-br/design-visualization/solutions/rendering/>

CUDA

Exemplos de uso real:

- Simulações de mecânica dos fluidos, física de partículas, previsão do tempo.



Arquitetura CUDA - Conceitos básicos

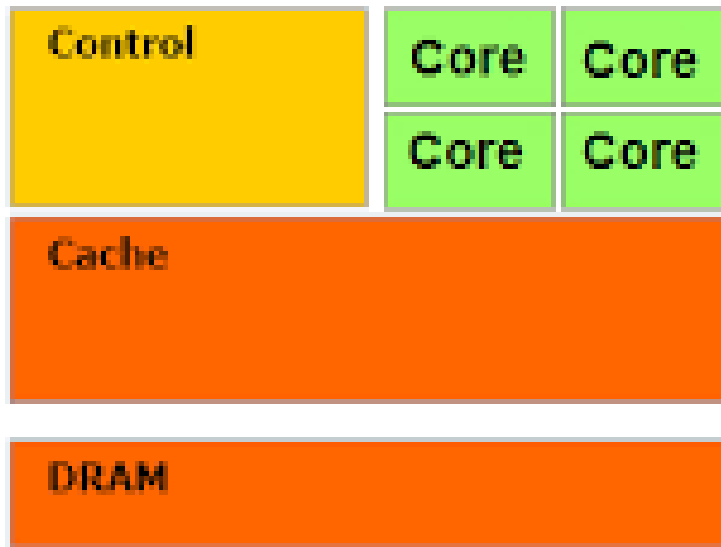
CUDA



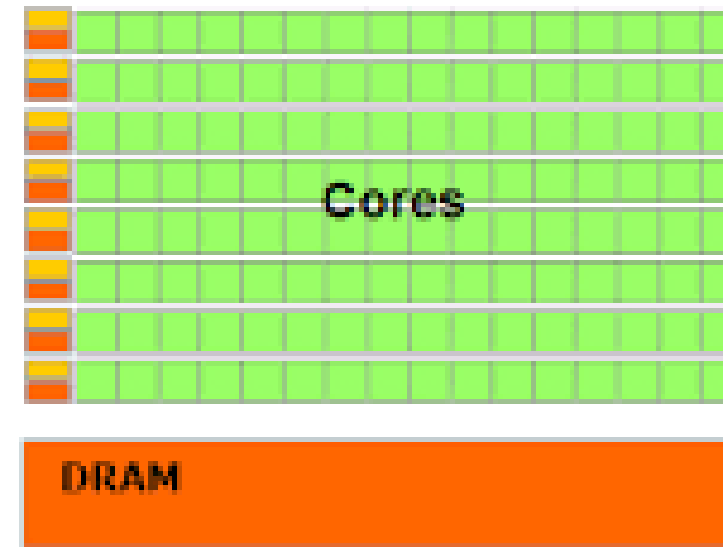
Modelo de execução:

- **Host (CPU):** onde o programa é iniciado e coordenado.
- **Device (GPU):** onde os kernels CUDA são executados.

<https://www.youtube.com/watch?v=sTSxamGenQo>



CPU



GPU



Estrutura da execução paralela:

- Um **kernel** é uma função que será executada por várias threads em paralelo.
- Threads são organizadas em **blocos (blocks)**, e os blocos são organizados em uma **grade (grid)**.

Grid

└─ Block[0]

└─ Thread[0], Thread[1], ..., Thread[n]

└─ Block[1]

└─ Thread[0], Thread[1], ..., Thread[n]

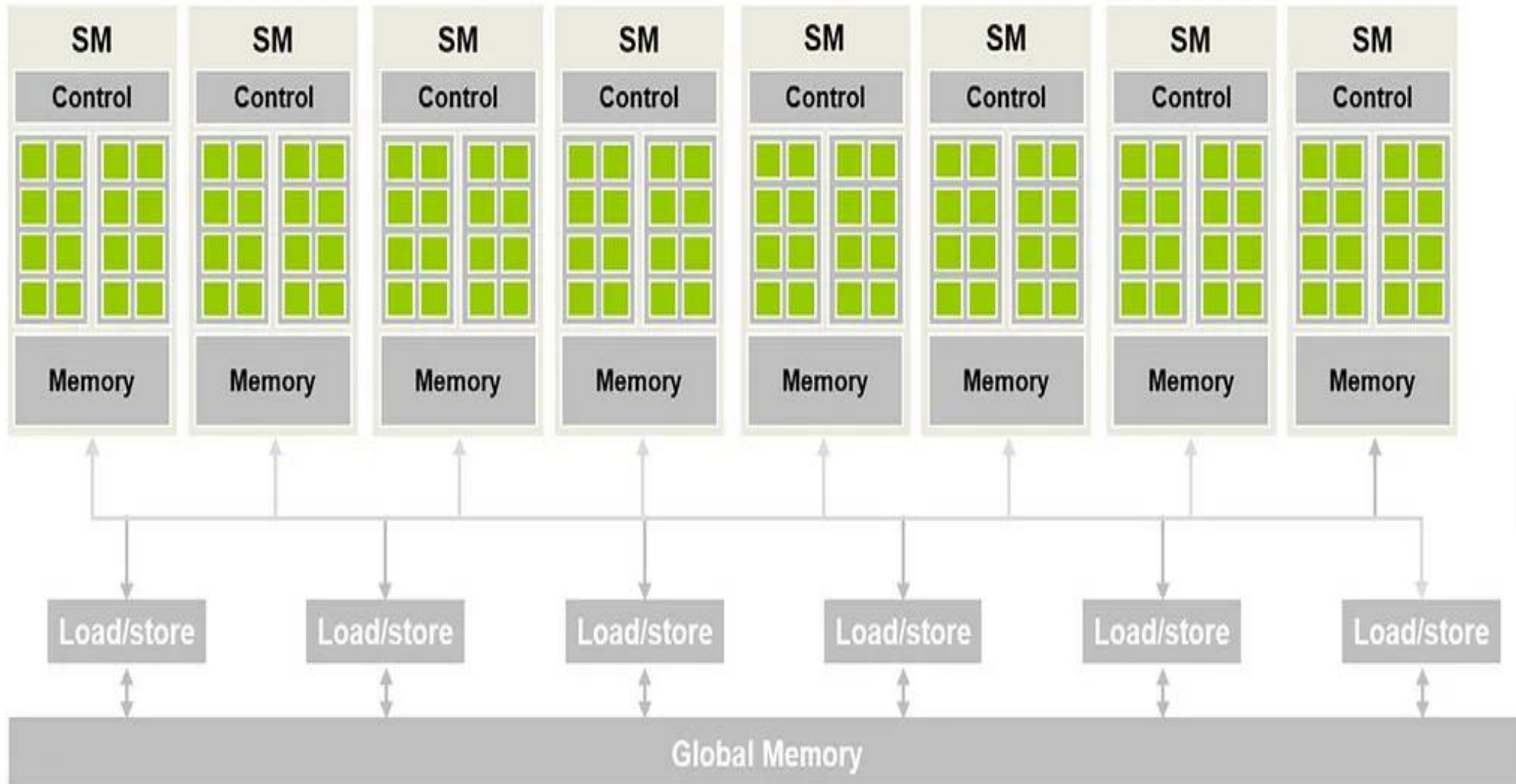


Tipos de memória:

- **Global memory:** acessível por todas as threads (mais lenta).
- **Shared memory:** compartilhada entre threads do mesmo bloco (mais rápida).
- **Registers e local memory:** uso individual da thread.



Tipos de memória



Exemplo prático – Soma de Vetores

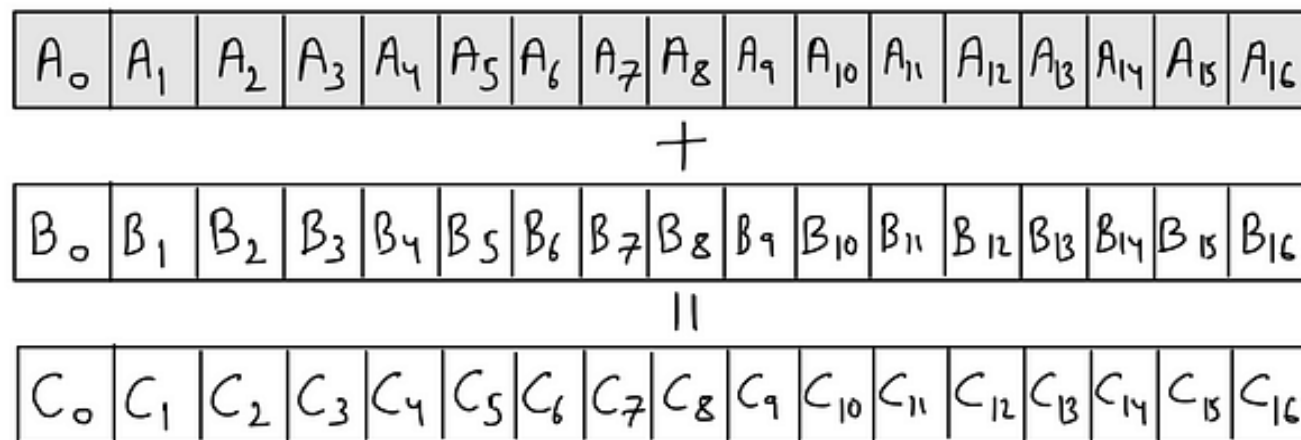
```
__global__ void somaVetores(int *a, int *b, int *c, int n) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Explicações:

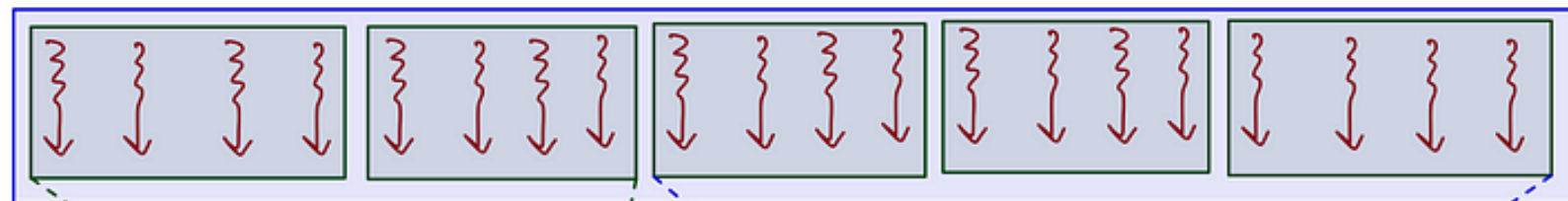
- `__global__` indica que a função roda na GPU e é chamada pela CPU
- `blockIdx.x` → índice do bloco atual.
- `threadIdx.x` → índice da thread dentro do bloco.
- `blockDim.x` → número de threads por bloco.



CUDA



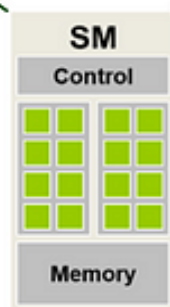
A grid of
20 threads



4 threads in
each block

2 blocks assigned
to this SM

3 blocks assigned to
this SM



Ferramentas e Desenvolvimento

Ferramentas e Desenvolvimento

- **Compilador:** `nvcc` (incluso no CUDA Toolkit)
- **IDEs:** Visual Studio, VS Code, CLion, Eclipse
- **Ferramentas úteis:**
- **Nsight Compute / Nsight Systems:** profiling e visualização de desempenho
- **cuda-memcheck:** checagem de acessos indevidos à memória
- **cuBLAS, cuFFT, cuDNN:** bibliotecas otimizadas pela NVIDIA



Exemplo prático – Soma de Vetores

```
__global__ void somaVetores(int *a, int *b, int *c, int n) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Explicações:

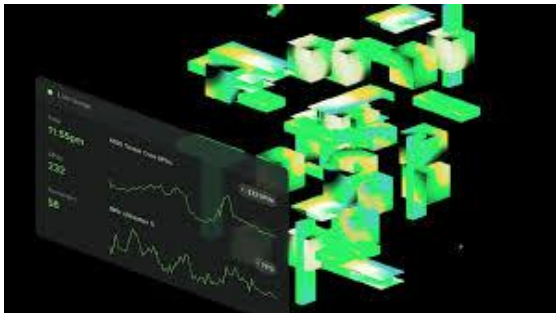
- `__global__` indica que a função roda na GPU e é chamada pela CPU
- `blockIdx.x` → índice do bloco atual.
- `threadIdx.x` → índice da thread dentro do bloco.
- `blockDim.x` → número de threads por bloco.



Etapas de um programa CUDA típico

1. Alocação de memória

```
cudaMalloc(&a_d, size);  
cudaMalloc(&b_d, size);
```



2. Cópia dos dados da CPU para GPU

```
cudaMemcpy(a_d, a_h, size, cudaMemcpyHostToDevice);
```



3. Execução do kernel

```
somaVetores<<<numBlocos, threadsPorBloco>>>(a_d, b_d, c_d, n);
```



4. Cópia dos resultados da GPU para CPU

```
cudaMemcpy(c_h, c_d, size, cudaMemcpyDeviceToHost);
```



5. Liberação de memória

```
cudaMemcpy(c_h, c_d, size, cudaMemcpyDeviceToHost);
```



Limitações e Desafios

Limitações e Desafios

- **Nem todo problema é paralelizável.**
- Transferências entre CPU e GPU podem causar gargalos.
- Debugging de código CUDA é mais complexo que em código C++ comum.
- **Requer placas NVIDIA compatíveis (com suporte a CUDA).**



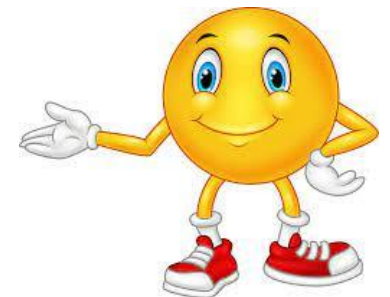
Ambientes de execução

CUDA

- Para programar usando CUDA on line, utilize o **Playground LeetGPU**

CUDA on-line neste link:

<https://leetgpu.com/playground>



Material para consulta/estudo de CUDA

Links para Estudo de CUDA

Para aprender mais sobre CUDA, acesse:

<https://www.youtube.com/watch?v=wyaPY8E3x30&t=5952s>

[https://www.revista-programar.info/static/downloads/Revista PROGRAMAR 38.pdf?t=site](https://www.revista-programar.info/static/downloads/Revista_PROGRAMAR_38.pdf?t=site)

DOCUMENTAÇÃO

<https://github.com/srush/GPU-Puzzles>

DEV BLOG

<https://developer.nvidia.com/blog/even-easier-introduction-cuda/>

TREINAMENTO CUDA GITHUB

<https://github.com/olcf/cuda-training-series>

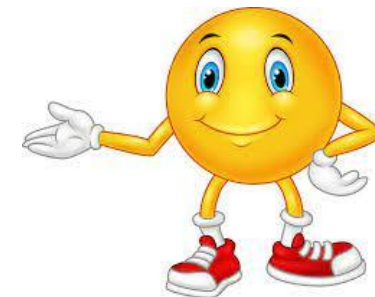
<https://github.com/srush/GPU-Puzzles>

TREINAMENTO OAK RIDGE

<https://www.olcf.ornl.gov/cuda-training-series/>

(tem relação com o primeiro link do github)

https://kernel-operations.io/keops/autodiff_gpus/what_is_a_gpu.html



Referências Bibliográficas

